



Technical Specification

Version: 1.0 · Status: Draft

Flutter · Dart · Swiss Ephemeris

Date: May 2026

1. Overview

Tithika is a tithi-focused Hindu lunar calendar app for iOS and Android. It computes tithis, nakshatras, festivals, and solar events using the Swiss Ephemeris astronomical engine, operating fully offline with no backend dependency. This document describes the architecture, module responsibilities, data flow, and implementation decisions for the v1 release.

Parameter	Value
App name	Tithika (तिथिका)
Platforms	iOS 16+ · Android 8+ (API 26)
Framework	Flutter 3.x (single codebase)
Language	Dart 3.x (null-safe)
Astronomy engine	Swiss Ephemeris via sweph ^3.2.1+2.10.3
State management	Riverpod 2.x (riverpod_generator code-gen)
Navigation	go_router 14.x
Persistence	shared_preferences
Location	geolocator 13.x + geocoding 3.x
Timezone	timezone 0.9.x + flutter_timezone 5.x
Operation mode	Fully offline — no network required
License	Open source (AGPL via Swiss Ephemeris)

2. High-Level Architecture

The app follows a layered architecture: UI → State → Services → Core engine. Riverpod providers wire these layers together reactively without tight coupling. All astronomical computation is synchronous and runs on an isolate-safe path (no async I/O after ephemeris initialisation).

Layer	Contents	Key files
UI / Features	Screens, widgets, navigation	lib/features/**
State	Riverpod providers & notifiers	lib/state/providers.dart lib/state/app_settings_notifier.dart
Services	Domain logic, festival detection, city search	lib/services/*.dart
Models	Immutable data classes	lib/models/*.dart
Core	Router, theme, time utilities	lib/core/*.dart
Native / FFI	Swiss Ephemeris via sweph package (Dart FFI)	sweph pub package

2.1 App Startup Sequence

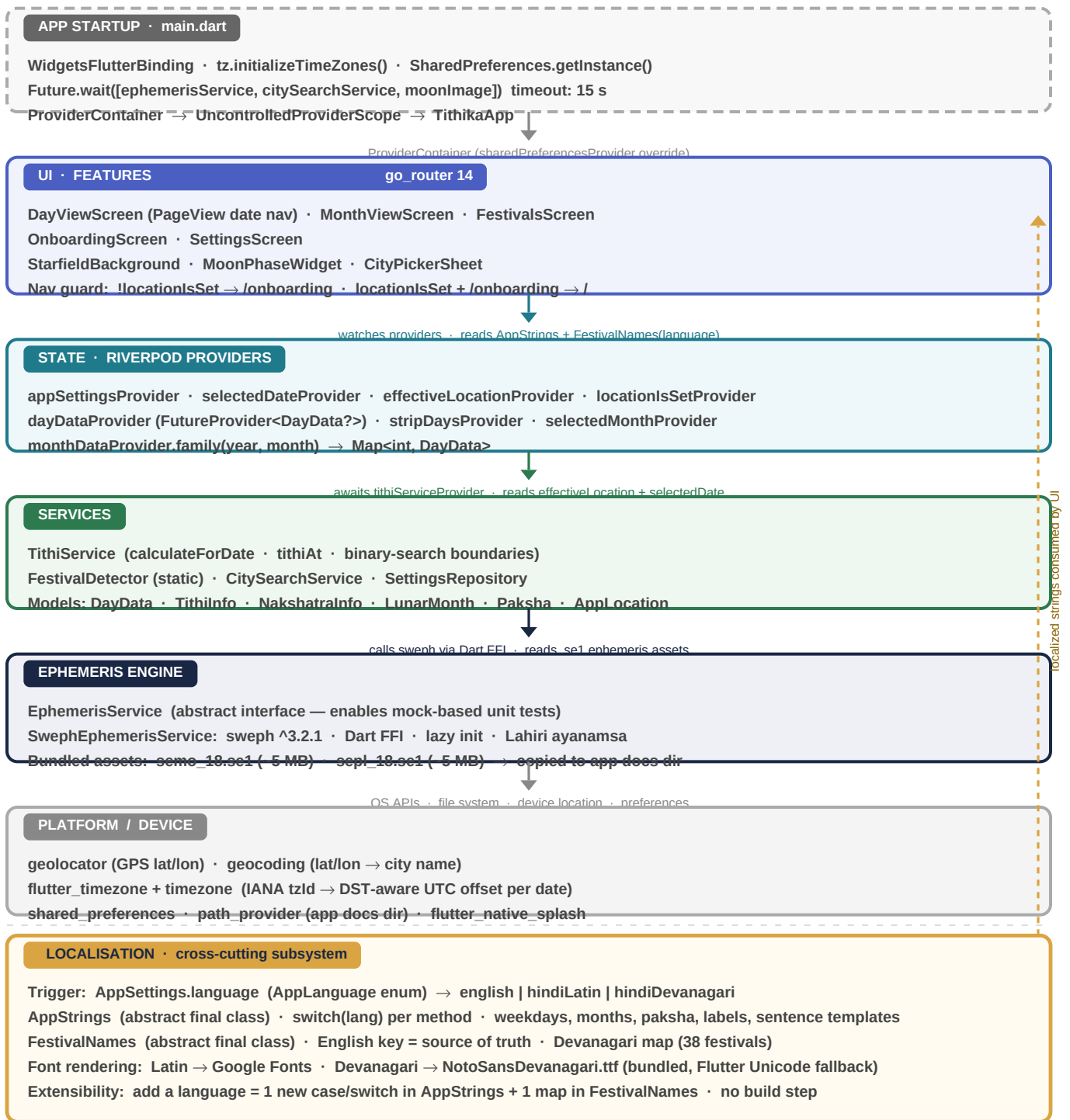
main() performs four steps before handing off to Flutter:

Step	Action	Detail
1	WidgetsFlutterBinding.ensureInitialized()	NativeSplash preserved; portrait orientation locked.
2	tz.initializeTimeZones()	Loads the full IANA timezone database into memory (timezone package).
3	SharedPreferences.getInstance()	Injected into ProviderContainer via sharedPreferencesProvider override.
4 (parallel)	Future.wait([ephemeris, cities, moonImage])	Pre-warms SwephEphemerisService (.se1 files copied to docs dir), loads cities dataset, precaches moon_circle.png. 15 s timeout guards against simulator hangs.

The result is an UncontrolledProviderScope wrapping TithikaApp. NativeSplash is removed on the first post-frame callback after Flutter renders, eliminating any blank-screen gap between the native splash and the app.

2.2 Architecture Overview

The diagram below shows all subsystems and the data-flow between them. Localisation is a cross-cutting subsystem highlighted at the bottom — it is driven by AppSettings.language and feeds localised strings directly into the UI layer.



2.3 Dependency Graph (simplified)

DayViewScreen → watches **dayDataProvider** → awaits **tithiServiceProvider** (which initialises **SweptEphemerisService**)
→ calls **TithiService.calculateForDate()** → calls **FestivalDetector.detect()** → returns **DayData**.

MonthViewScreen → watches **monthDataProvider(year, month)** (FutureProvider.family) → same service path,
computing all days in the month in a single sync loop.

3. Swiss Ephemeris Integration

3.1 Package

The **sweph** pub package (^3.2.1+2.10.3) wraps the C Swiss Ephemeris library via Dart FFI. It is initialised once at app startup and kept alive for the session. The ephemeris data files (~10 MB) are bundled as Flutter assets and copied to the app documents directory on first use.

3.2 Initialisation

```
await Sweph.init(  
  epheAssets: ['assets/ephemeris/semo_18.se1', 'assets/ephemeris/sepl_18.se1'],  
  assetLoader: _FlutterAssetLoader(), // rootBundle → Uint8List  
  epheFilePath: '${docsDir.path}/ephe',  
);  
Sweph.swe_set_sid_mode(SiderealMode.SE_SIDM_LAHIRI); // Lahiri ayanamsa
```

The **_FlutterAssetLoader** shim reads asset bytes via `rootBundle.load()` and returns a `Uint8List` as required by the `sweph` initialiser.

3.3 Calculation Flags

Flag set	Constant	Used for
Tropical	SEFLG_SWIEPH	Sun & Moon longitude for tithi/elongation
Sidereal (Lahiri)	SEFLG_SWIEPH SEFLG_SIDEREAL	Nakshatra, sun zodiac sign, lunar month

3.4 Sunrise / Sunset

Computed via `swe_rise_trans()` with standard atmosphere (1013.25 mbar, 15 °C). The `GeoPosition` argument takes **longitude first, latitude second** (note: the opposite of most mapping APIs). Sunrise is the primary reference time for the Drik day-boundary rule; if GPS is unavailable, UTC noon is used as a safe fallback.

4. Core Calculations

4.1 Tithi

A tithi is 1/30th of a synodic month — one of 30 equal arcs of the Moon–Sun elongation:

```
elongation = (moonLong - sunLong + 360) % 360  
tithiNumber = ceil(elongation / 12) // 1-30  
// Special case: at exactly 0° elongation, ceil() = 0 → clamp to 30 (Amavasya)
```

Tithis 1–15 are **Shukla Paksha** (waxing). Tithis 16–30 are **Krishna Paksha** (waning), displayed as Krishna 1–14 + Amavasya.

Tithi boundary binary search

Start and end times are found by bisecting the elongation function within a 1.5-day window (52 iterations → sub-second accuracy, $\sim 0.00003^\circ \approx < 1$ min). The convergence criterion is the standard "d > 180° → not yet crossed" heuristic, which is safe within the 1.5-day window.

4.2 Nakshatra

```
nakshatraNumber = floor(siderealMoonLong / (360/27)) + 1 // 1-27
endTime found by bisecting sidereal Moon longitude to next 13.333...° boundary
```

Sidereal Moon longitude uses the Lahiri ayanamsa, consistent with mainstream Indian ephemeris practice.

4.3 Lunar Month (Amanta base) + Adhika Maas Detection

TithiService._lunarMonthAndAdhika() returns both the lunar month name and a boolean isAdhika flag. It uses the **Amanta** definition as its base: the month runs from one Amavasya (new moon, elongation = 0°) to the next.

Step 1 — Locate previous and next Amavasya

```
// Elongation at sunrise JD
curElong = (moonLong - sunLong + 360) % 360

// Days elapsed since previous Amavasya (approx)
daysSincePrev = curElong / 12.19

// Binary-search in a 5-day window starting ~2 days before the estimate
prevAmavasyaJd = _nextBoundaryJd(
  sunriseJd - daysSincePrev - 2.0,
  targetAngle: 0.0, // Amavasya = elongation 0°
  maxDays: 5.0
)

// Days remaining to next Amavasya
daysToNext = (360 - curElong) / 12.19
nextAmavasyaJd = _nextBoundaryJd(
  sunriseJd + daysToNext - 2.0,
  targetAngle: 0.0,
  maxDays: 5.0
)
```

Step 2 — Check for Adhika (intercalary) month

A month is **Adhika** if the sun does **not** transit into a new sidereal rashi (zodiac sign) between the two consecutive Amavasyas that bracket it. Equivalently: both Amavasyas fall in the same rashi.

```
signAtPrev = floor(siderealSunLong(prevAmavasyaJd) / 30) // 0-11
signAtNext = floor(siderealSunLong(nextAmavasyaJd) / 30)

isAdhika = (signAtPrev == signAtNext) // no Sankranti between new moons
```

Winter months: implicit exclusion via orbital mechanics

There is no explicit blacklist of months that cannot be Adhika. Winter exclusion is guaranteed by the sun's variable orbital speed:

Period	Sun speed (approx.)	Days per sign	Adhika possible?
Near perihelion (Dec–Feb)	~1.02°/day	~29.0 days	No — faster than one synodic month (29.53 d)
Average	~0.986°/day	~30.4 days	Marginal — Phalguna Adhika is extremely rare
Near aphelion (May–Jul)	~0.953°/day	~31.5 days	Yes — Jyeshtha/Ashadha/Shravana most common

Because the sun traverses each winter sign in fewer days than a synodic month, there will always be at least one sign change between two consecutive Amavasyas in those months. The algorithm therefore produces $\text{signAtPrev} \neq \text{signAtNext} \rightarrow \text{isAdhika} = \text{false}$ for all winter months without any additional logic. Traditional sources that list "months which cannot be Adhika" are enumerating the same astronomical fact, not an independent rule that needs to be coded separately.

Step 3 — Assign month name

The month name is always the sidereal sun sign at the **previous** Amavasya (Amanta base). `LunarMonth.values[signAtPrev]` maps sign index to month (0 = Mesha → Chaitra ... 11 = Meena → Phalguna). This formula is correct for regular months, Adhika months, and Nija months (the regular month that immediately follows an Adhika month).

`DayData.isAdhika` is stored alongside the month. The UI prepends the localised "Adhika" prefix (`AppStrings.adhikaPrefix(language)`) to the month name when this flag is true.

4.4 Purnimanta Month System Correction

The **Purnimanta** system (North Indian convention) ends each month at Purnima rather than Amavasya, so Krishna Paksha days are assigned to the *following* month compared to Amanta. The correction is applied in `_applyMonthSystem()` in the UI layer (not in `TithiService`):

```
DayData _applyMonthSystem(DayData raw, MonthSystem system) {
    if (system == MonthSystem.amanta) return raw; // no change
    // Purnimanta: advance month by 1 for Krishna Paksha non-Adhika days
    if (!raw.isAdhika && raw.tithi.paksha == Paksha.krishna) {
        final next = LunarMonth.values[(raw.lunarMonth.index + 1) % 12];
        return raw.copyWith(lunarMonth: next);
    }
    return raw; // Shukla Paksha and Adhika days are unchanged
}
```

Important: Adhika months are *never* shifted — an Adhika Krishna Paksha day keeps its Adhika month label under both systems. Festival detection always uses the Purnimanta form regardless of the user's month-system setting (see §8.1).

4.5 Kshaya (Skipped) Tithi Detection

If a tithi's end time falls before the next day's sunrise, a second tithi begins within the same calendar day. `DayData.secondaryTithi` carries this when present. Most days it is null.

4.6 Solar Festivals (Sankranti)

Detected by comparing the sidereal sun zodiac sign (`floor(siderealSunLong / 30)`) between the previous sunrise and the current sunrise. `DayData.sunZodiacEntryToday = true` flags a Sankranti day. Makar Sankranti (sign 9) and Baisakhi/Vishu (sign 0) are the two v1 solar festivals.

5. Data Models

Model	Fields	Notes
DayData	localDate, tithi, nakshatra, sunriseUtc, sunsetUtc, lunarMonth, sunZodiacSign, sunZodiacEntryToday, festivalName, secondaryTithi	Central aggregate for a calendar day. Immutable; copyWith() for festival/secondary tithi.
TithiInfo	number (1-30), pakshaNumber, paksha, start, end (UTC), special	start/end are UTC DateTimes. special is SpecialTithi? enum.
NakshatraInfo	number (1-27), end (UTC)	Name resolved from number via l10n layer.
AppSettings	location, language, monthSystem	Persisted via shared_preferences through SettingsRepository.
AppLocation	lat, lon, displayName, tzId	tzId used to resolve DST-aware UTC offset via flutter_timezone. Static Ujjain constant as fallback.
LunarMonth	enum (Chaitra ... Phalguna, 12 values)	Index matches sidereal sun sign at Purnima (0=Mesha→Chaitra).
Paksha	enum: shukla, krishna	
SpecialTithi	enum: ekadashi, purnima, amavasya, vinayakaChaturthi, sankashtiChaturthi, pradosh	Used for indicator dots in calendar grid.

6. State Management

The app uses **flutter_riverpod 2.x** with code generation via **riverpod_generator** (build_runner). All providers are defined in `lib/state/providers.dart`.

Provider	Type	Responsibility
appSettingsProvider	NotifierProvider	Reads/writes AppSettings. Backed by SettingsRepository (shared_preferences).
selectedDateProvider	StateProvider	The date shown in Day View. Updated by strip tap or month grid tap.
effectiveLocationProvider	Provider	Returns settings.location ?? AppLocation.ujjain — ensures calculations always have a valid location.
locationIsSetProvider	Provider	True once user has explicitly set a location. Drives GoRouter redirect to onboarding.
dayDataProvider	FutureProvider	Awaits tithiServiceProvider, then calls TithiService.calculateForDate(). Null while ephemeris loads.

stripDaysProvider	FutureProvider >	4-day strip: [selected-1, selected, selected+1, selected+2].
selectedMonthProvider	StateProvider	Month displayed in MonthViewScreen (year+month only).
monthDataProvider	FutureProvider.family , (int,int)>	Full DayData for every day in (year, month), keyed by day-of-month. Festival names included.

6.1 AppSettingsNotifier

A Notifier<AppSettings> that proxies reads/writes through SettingsRepository. On build(), it loads settings synchronously from a pre-populated cache (SettingsRepository initialises the shared_preferences instance during service startup). Location, language, and month system changes each call the appropriate repository setter and emit a new state.

7. Navigation

Navigation uses **go_router 14.x**, built inside a ConsumerWidget so the redirect logic can read Riverpod providers.

Route	Path	Screen	Guard
Onboarding	/onboarding	OnboardingScreen	Redirected here if !locationIsSet
Day View	/	DayViewScreen	Redirected away if locationIsSet && going to /onboarding
Month View	/month	MonthViewScreen	Child of Day View
Settings	/settings	SettingsScreen	Child of Day View

The `_ProviderListenable` bridge makes `GoRouter.refreshListenable` react to `locationIsSetProvider` changes, so the router re-evaluates its redirect whenever the user completes onboarding.

8. Feature Modules

8.1 Day View (lib/features/day_view/)

File	Responsibility
day_view_screen.dart	Root screen. Uses a PageView for infinite horizontal date navigation. Each page index maps to a calendar date via a fixed UTC epoch (2000-01-01 + offset of 100 000), keeping DST-safe integer arithmetic. Watches dayDataProvider and stripDaysProvider. Composes top 75% (hero panel) and bottom 25% (4-day strip). Handles date selection from strip or month grid via selectedDateProvider. Also applies Purnimanta/Amanta month-system correction (<code>_applyMonthSystem</code>) before rendering.
moon_phase_widget.dart	Programmatic moon phase renderer. Derives illumination from tithi number: $illumination = \cos(\pi \times tithiNumber / 15) $. Draws a stylised dark/light hemisphere.

location_banner.dart	Persistent "Tap to set your location" banner shown when effectiveLocation is the Ujjain fallback. Navigates to Settings on tap.
----------------------	---

8.1.1 Day View Render Pipeline

Each page in the PageView builds a `_DayPageContent` widget for its date. The render pipeline executes the following steps in order:

Step	Operation	Detail
1	Resolve date from page index	pageIndex → date: UTC epoch 2000-01-01 + (page – 100,000) days. UTC avoids DST off-by-one errors at midnight boundaries.
2	TithiService.calculateForDate()	Computes raw DayData: tithi, nakshatra, sunrise/sunset, lunarMonth, sunZodiacSign, isAdhika, secondaryTithi. Uses location lat/lon and DST-aware tzOffset for the specific date.
3	Apply month system correction	<code>_applyMonthSystem(raw, monthSystem)</code> : if Purnimanta + Krishna Paksha + !isAdhika → advance lunarMonth index by 1. Adhika days are never shifted.
4	Festival detection (always Purnimanta)	<code>FestivalDetector.detect()</code> is called on the Purnimanta-adjusted data — regardless of the user's month-system setting. Ensures festival labels always match Purnimanta calendar sources.
5	Moon phase illumination	$\text{illumination} = \cos(\pi \times \text{tithiNumber} / 15) $. Tithi 1 ≈ 0.02 (thin crescent), tithi 8 = 0.5 (quarter), tithi 15 = 1.0 (full), tithi 30 ≈ 0 (new/dark). Rendered by <code>MoonPhaseWidget</code> as a programmatic hemisphere.
6	Lunar month label	adhikaPrefix (if isAdhika) + monthName + " . " + pakshaUpper + " " + pakshaWord. All strings localised via <code>AppStrings</code> using <code>AppSettings.language</code> .
7	Tithi window	tithi.start and tithi.end (UTC) converted to local time using <code>location.tzOffsetAt(date)</code> . Displayed as "HH:MM AM – HH:MM PM".
8	Secondary tithi	If <code>DayData.secondaryTithi != null</code> , <code>AppStrings.secondaryTithiBegins()</code> renders "{name} begins {time}" (EN) or "{name} {time} ■■■" (Devanagari) with paksha colour.
9	Nakshatra card	Nakshatra name (localised) + <code>AppStrings.nakshatraUntil(endTime)</code> . End time uses same tzOffset conversion.
10	Sunrise / sunset card	sunriseUtc and sunsetUtc converted to local time. Falls back to "—" if <code>swe_rise_trans()</code> returned null (GPS failure or polar edge case).
11	Festival badge	If <code>festivalName != null</code> : <code>FestivalNames.localize(key, language)</code> renders a rounded pill in saffron. English key is always the source; Devanagari map lookup applied for hindiDevanagari.
12	isToday check	Computed as: <code>localNow = DateTime.now().toUtc().add(location.tzOffset)</code> , then date comparison. Uses location timezone — not device local time — so "Today" is correct for users in a different timezone from their selected city.

4-Day Strip render logic

The strip shows [selected – 1, selected, selected + 1, selected + 2]. Each cell displays: weekday short, Gregorian day number, a paksha colour dot + tithi pakshaNumber. Label priority for the bottom row:

- **Festival name** (from DayData.festivalName) — rendered in saffron
- **Ekadashi** (tithi 11 or 26) / **Purnima** (tithi 15) / **Amavasya** (tithi 30) — rendered in paksha colour
- No label otherwise

The selected cell is highlighted with a top gold border + pale shukla background. Tapping any cell sets selectedDateProvider, which causes both the PageView to jump to the new page and selectedMonthProvider to update if the month changed.

8.2 Month View (lib/features/month_view/)

MonthViewScreen watches monthDataProvider(year, month). Renders a 7-column calendar grid. Each cell shows: Gregorian date, tithi number (sunrise-prevailing), small moon phase icon, paksha colour indicator, and a dot for special tithis / festivals. Tapping a cell sets selectedDateProvider and navigates back to Day View.

8.3 Onboarding (lib/features/onboarding/)

Displayed on first launch (or whenever locationIsSetProvider is false). Two flows:

- **Use current location** — calls geolocator for GPS permission, captures lat/lon, resolves city name via geocoding.
- **Enter manually** — launches CityPickerSheet, a searchable bottom sheet backed by CitySearchService.

On completion, saves an AppLocation to appSettingsProvider, which triggers the GoRouter redirect to Day View.

8.4 Settings (lib/features/settings/)

Three settings sections: Location (GPS refresh + city picker), Language (English / Hindi Latin / Hindi Devanagari), Month system (Purnimanta / Amanta). Each change calls the appropriate AppSettingsNotifier method, which persists to shared_preferences and emits a new state, reactively updating all dependent providers.

8.5 Festivals Screen (lib/features/festivals/)

FestivalsScreen provides a dedicated list view of upcoming festivals. It uses FestivalDetector to scan forward from the current date and renders festival names localised via FestivalNames.localize(). Navigation is wired through go_router as a child route of the Day View.

8.6 Shared Widgets (lib/features/shared/)

Widget	Description
CityPickerSheet	Searchable bottom sheet. Queries CitySearchService with debounce. Displays city name + country; on selection, returns AppLocation.
StarfieldBackground	Animated starfield drawn on a CustomPainter using a seeded Random for star positions. Used across all screens to create the dark night-sky aesthetic.

9. Services

9.1 EphemerisService (interface)

`lib/services/ephemeris_service.dart` defines the abstract interface: `initialize()`, `julianDayFromUtc()`, `utcFromJulianDay()`, `sunLongitude()`, `moonLongitude()`, `siderealSunLongitude()`, `siderealMoonLongitude()`, `sunrise()`, `sunset()`. This abstraction allows unit testing via a mock/stub implementation without loading the real ephemeris files.

9.2 SwephEphemerisService

The production implementation (`lib/services/sweph_ephemeris_service.dart`). Wraps the `sweph` package. Uses lazy initialisation — `initialize()` is idempotent (early-returns if already initialised). All public methods call synchronous `sweph` functions after initialisation.

9.3 TithiService

The core domain service (`lib/services/tithi_service.dart`). Accepts an `EphemerisService` dependency. Provides:

- `calculateForDate(localDate, lat, lon, tzOffset) → DayData` (full computation for one day)
- `tithiAt(utcDateTime) → TithiInfo` (live tithi for Day View header)

9.4 FestivalDetector

`lib/services/festival_detector.dart` — a pure static class that maps `DayData` to a festival name string (or null). Implements two detection strategies:

- **Tithi-based:** matches (`lunarMonth`, `tithi.number`) against a lookup table of 36 festivals.
- **Solar:** checks `sunZodiacEntryToday` and `sunZodiacSign` for Makar Sankranti (sign 9) and Baisakhi/Vishu (sign 0).

9.5 CitySearchService

Loads a bundled cities dataset from `assets/cities/`. Provides fuzzy prefix search returning `AppLocation` results. Used by the city picker sheet in Onboarding and Settings.

9.6 SettingsRepository

Wraps `shared_preferences` to persist and retrieve `AppSettings` fields. Encodes `AppLocation` as a JSON string. Provides typed getters/setters for each setting field.

10. Localisation (l10n)

Tithika uses a **custom static localisation approach** — not Flutter's gen-l10n / ARB system. All translatable strings are handled by two abstract final classes in `lib/core/`. The active language is `AppSettings.language` (`AppLanguage` enum), persisted via `SettingsRepository` and read from `appSettingsProvider`.

Enum value	Display	Script
------------	---------	--------

AppLanguage.english	English	Latin — Google Fonts
AppLanguage.hindiLatin	Hindi (Latin script)	Latin — same as English; festival names may differ (e.g. "Deepavali")
AppLanguage.hindiDevanagari	Hindi (Devanagari)	देवनागरी — NotoSansDevanagari.ttf

10.1 AppStrings (lib/core/app_strings.dart)

An abstract final class — no instances, all static methods. Every UI label that varies by language lives here. Each method performs a switch (lang): the hindiDevanagari case returns from a const data list; all other cases return the English form. This means **hindiLatin currently renders identically to English** — distinct Latin transliterations can be added per-method without changing any other file.

Method / data	EN example	Devanagari example
weekdayFull(weekday, lang)	MONDAY	सोमवार
weekdayShort(weekday, lang)	MON	सोम
weekdayLetter(sundayFirst, lang)	M	सो
gregMonth(month, lang)	JANUARY	जनवरी
gregMonthShort(month, lang)	Jan	जन
pakshaUpper(paksha, lang)	SHUKLA	शुक्ल
paksha(paksha, lang)	Shukla	शुक्ल
pakshaWord(lang)	PAKSHA	पक्ष
nakshatra(lang)	NAKSHATRA	नक्षत्र
sunrise(lang)	SUNRISE	सूर्योदय
sunset(lang)	SUNSET	सूर्यास्त
ekadashi / purnima / amavasya	Ekadashi ...	एकादशी ... पूर्णमा ... अमावस्या
nakshatraUntil(time, lang)	"until 8:17 PM"	"तक 8:17 PM"
hinduMonthBegins(...)	Vaishakha begins Mon, May 12 at 6:30 AM	वैशाख सोमवार, मई 12 को 6:30 AM से
secondaryTithiBegins(...)	· Dwitiya begins 7:45 PM	· द्वितीया 7:45 PM से

Sentence templates (hinduMonthBegins, secondaryTithiBegins, nakshatraUntil) handle Hindi word-order differences explicitly — the Devanagari case rearranges the interpolated arguments rather than just substituting words in English order.

10.2 FestivalNames (lib/core/festival_names.dart)

An abstract final class with a single static method localize(String? key, AppLanguage language) → String?. The **English name is the canonical key** produced by FestivalDetector.detect(). For English and hindiLatin, the key is returned as-is. For

hindiDevanagari, the key is looked up in a private const Map<String, String> _deva; if the key is missing the English fallback is used.

English key (FestivalDetector output)	Devanagari
Diwali	दीपावली
Maha Shivaratri	महाशिवरात्रि
Krishna Janmashtami	कृष्ण जन्माष्टमी
Ganesh Chaturthi	गणेश चतुर्थी
Makar Sankranti / Pongal	मकर संक्रांति / पोंगल
Baisakhi / Vishu	बैसाखी / वसु
... 32 more festival keys	(full map in festival_names.dart)

10.3 Font Rendering

Latin script is rendered via **Google Fonts** (google_fonts 6.2.1) — fonts are loaded at runtime. Devanagari script uses **NotoSansDevanagari.ttf**, bundled as a Flutter asset (assets/fonts/) and declared in pubspec.yaml under the NotoSansDevanagari font family. Flutter's text rendering selects fonts by Unicode block automatically — no per-character font switching is needed in widget code.

10.4 Extensibility

Adding a new language requires changes in exactly two files:

- Add a new enum value to AppLanguage in lib/models/app_settings.dart.
- Add one new case per switch in AppStrings, backed by a new const data list.
- Add one new Map<String, String> in FestivalNames.
- Add a font asset if the new language uses a non-Latin script.

No generated code, no ARB files, no build step required for localisation changes.

11. Visual Theme

Dark night-sky aesthetic across all screens. Defined in lib/core/theme.dart. Starfield background rendered by the StarfieldBackground widget on every screen.

Token	Hex	Usage
Navy	#1A2744	Primary background, app bars, card headers
Indigo	#4A5FC1	Shukla Paksha accent, interactive elements
Gold	#D9A441	Shukla highlight, festival labels, accent stripes
Saffron	#C8421D	Festival indicators, error states

Deep space	#050714	Background base / splash screen background
------------	---------	--

Typography: Google Fonts (google_fonts 6.2.1) for Latin script. NotoSansDevanagari bundled locally for Devanagari text rendering. Typography scale and open items (exact Devanagari pairing) to be finalised during design phase.

12. Asset Inventory

Asset path	Contents	Size (approx.)
assets/ephemeris/semo_18.se1	Swiss Ephemeris Moon data file	~5 MB
assets/ephemeris/sepl_18.se1	Swiss Ephemeris Sun/planet data file	~5 MB
assets/cities/	Cities dataset for city picker search	~2 MB
assets/fonts/NotoSansDevanagari.ttf	Devanagari script font	~1 MB
assets/icon.png	App icon (used by flutter_launcher_icons)	<0.1 MB
assets/moon_circle.png	Moon shape mask / overlay	<0.5 MB
assets/moon.jpg	Moon texture image	<1 MB
assets/launch_moon.png	Splash screen moon image	<0.5 MB

Total installed app target: **<30 MB**. Ephemeris files are copied to the app documents directory at first launch; subsequent launches skip this step.

13. Non-Functional Requirements

Requirement	Target	Implementation notes
Accuracy	Tithi transitions within 1 min of published Drik Panchang	52-iteration binary search on Swiss Ephemeris positions yields ~0.00003° accuracy.
Offline	100% — no network required	Ephemeris files bundled. No analytics or remote calls in v1.
Performance	Main screen renders <200 ms (mid-range Android)	All tithi/nakshatra math is synchronous and CPU-cheap after ephe init. Month data computed eagerly per day.
App size	<30 MB installed	Dominated by ephemeris (~10 MB) + fonts (~1 MB). No unnecessary asset bloat.
Privacy	Location stays on-device	geolocator captures lat/lon only. No analytics, no crash reporting in v1.

Accessibility	Minimum contrast ratios met	Pending design review — colour palette v1 draft being finalised.
---------------	-----------------------------	--

14. Testing Strategy

14.1 Unit Tests

All tests live in test/. The EphemerisService abstraction enables unit testing of TithiService and FestivalDetector without loading real ephemeris data. A MockEphemerisService returns fixed planetary positions for known dates.

Key test scenarios:

- Tithi boundary dates: verify Purnima on published dates (e.g. 2024-09-17 Purnima)
- Kshaya tithi: verify secondaryTithi is populated when end < next sunrise
- Amavasya edge case: elongation at exactly 0° → tithi 30 (not 0)
- Lunar month: verify Chaitra assigned correctly for post-Holi dates
- Festival detection: spot-check 10+ festivals against known 2025/2026 dates
- Solar festivals: Makar Sankranti 2025-01-14, Baisakhi 2025-04-14

14.2 Widget Tests

Moon phase widget: verify correct illumination for tithi 1 (thin crescent), tithi 15 (full), tithi 30 (new/dark). Month grid: verify correct tithi numbers rendered for a known month. Strip: verify 4 days rendered with correct date offsets.

14.3 Integration / Acceptance Tests

Manual spot-checks against Drik Panchang website for current month. Automated integration tests are out of scope for v1 (ephemeris file access in CI requires platform channels).

15. Build & Distribution

15.1 Toolchain

Tool	Version	Notes
Flutter SDK	3.x (stable)	Pin to a specific version in .fvmrc (recommended)
Dart SDK	^3.11.5	Set in pubspec.yaml environment.sdk
build_runner	^2.4.0	Code generation for Riverpod annotations
flutter_launcher_icons	^0.14.3	Generates iOS/Android icons from assets/icon.png
flutter_native_splash	^2.4.3	Generates native splash screen (bg #050714 + moon image)

15.2 Code Generation

Run `dart run build_runner build --delete-conflicting-outputs` after modifying any file annotated with `@riverpod`. Generated files are committed to source control (not gitignored).

15.3 Release Targets

Platform	Command	Notes
Android	<code>flutter build appbundle --release</code>	AAB uploaded to Play Console. minSdk 26.
iOS	<code>flutter build ipa --release</code>	IPA via Xcode Archive → App Store Connect. iOS 16+.

16. Open Items

#	Item	Status
1	Colour palette finalisation (Shukla/Krishna exact values, festival indicators)	Design phase
2	Typography — exact Devanagari font pairing and size scale	Design phase
3	Moon image: static vs. animated phase transition	TBD
4	Amanta month system implementation in TithiService	v1 if time allows
5	City dataset source and update strategy	TBD
6	Crash reporting / analytics strategy for post-launch monitoring	v2
7	CI/CD pipeline setup (GitHub Actions + code signing)	Pre-launch

17. Out of Scope (v1)

- Yoga, Karana (remaining Panchanga elements)
- Rahu Kaal, Yamaganda, Gulika, Abhijit muhurta
- Moonrise / moonset times
- Push notifications (Ekadashi, Purnima, festivals)
- Home-screen widgets (iOS / Android)
- Vakya calculation method
- Regional calendar variations (Tamil, Bengali, Malayalam, Marathi, Gujarati)
- Multi-region festival date variants
- Photo-realistic moon imagery
- Monetisation (ads, in-app purchase, paid tier)
- Analytics or crash reporting
- Muhurta / auspicious time calculations